

Document Scanning and Enhancement: From Raw Photo to Clean Scan

Course Project - Computer Vision

Student Information

- **Name:** Ali Kashi Pazha
- **Student ID:** 40121723
- **GitHub Repository:** <https://github.com/alikashipazha/Computer-Vision-Project>
- **Google Colab:** <https://colab.research.google.com/drive/13TLa1VMpKMHIKJZekUlmHlsiSRP5Rq0h?usp=sharing>

Academic Information

- **University:** K. N. Toosi University of Technology
- **Faculty:** Faculty of Computer Engineering
- **Course:** Computer Vision
- **Instructor:** Dr. Nasihatkon

Abstract

This project implements an end-to-end document scanning and enhancement pipeline. It addresses the common computer vision task of image-to-image translation by converting degraded, shadowed, and perspective-distorted smartphone photographs of documents into crisp, uniform, scan-quality digital copies. The system is designed around two decoupled components: a **Four-Corner Detection Network** and a **Document Enhancement Network**. This report outlines the engineering design, mathematical foundations, and pipeline implementations developed during the first two phases of the project.

1. Phase 1: Dataset Preparation

Rather than manually annotating thousands of training images, this project leverages a data-centric deep learning paradigm: **generating a supervised dataset synthetically**. By mathematically transforming clean document scans onto arbitrary background images, we obtain high-fidelity training pairs along with pixel-perfect ground truth labels without human annotation.

1.1. Real-World Test Set and Annotation

A real-world test set of 10–15 diverse smartphone photographs was collected to evaluate the model’s generalization capabilities on entirely unseen documents (such as hand-written notes, textbook pages, and printed homework). - **Commercial Baseline:** For each test photo, a corresponding high-quality reference scan was acquired using a commercial scanning application (e.g., CamScanner). This serves as the target commercial baseline during final evaluations. - **Keypoint Annotation:** The four corners of the document in each raw smartphone photo were manually labeled using Roboflow. A strict, consistent ordering of keypoints was maintained: 1. **Top-Left (TL)** 2. **Bottom-Left (BL)** 3. **Bottom-Right (BR)** 4. **Top-Right (TR)**

Keypoint Index	Corner Name	Target Orientation	Consistent Ordering Rule
1	Top-Left (TL)	(0, 0)	Must be the first annotated point across all samples

Keypoint Index	Corner Name	Target Orientation	Consistent Ordering Rule
2	Bottom-Left (BL)	$(0, H - 1)$	Must be the second annotated point
3	Bottom-Right (BR)	$(W - 1, H - 1)$	Must be the third annotated point
4	Top-Right (TR)	$(W - 1, 0)$	Must be the fourth annotated point

Consistent ordering is mathematically critical to ensure the correctness of subsequent perspective rectification transforms.

1.2. Synthetic Training Data Generation

The synthetic training pair generator (`src/dataset/generator.py`) simulates the physical pipeline of taking a photograph of a document placed on a surface:

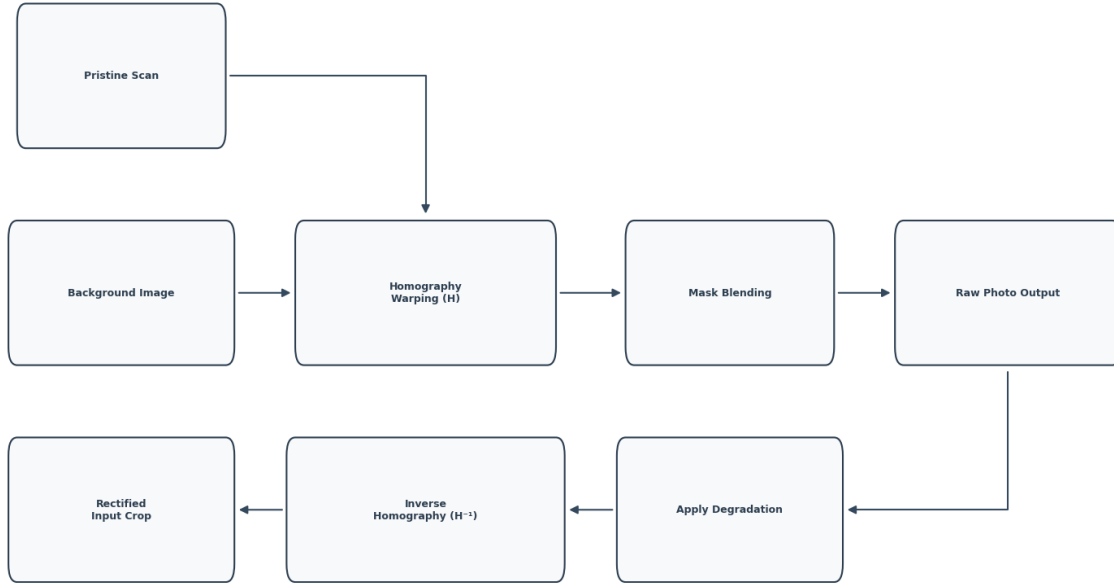


Figure 1: Flowchart of the Synthetic Training Data Generation pipeline.

1. **Clean Scan Selection:** A pristine, high-resolution document scan is selected as the ground truth target.
2. **Background Selection:** A random textured background image (e.g., table, desk, carpet) is loaded.
3. **Quadrilateral Generation:** A randomized convex quadrilateral is generated within the boundaries of the background image to represent the target document’s position and camera perspective distortion.
4. **Homography Warping:** The perspective transform matrix $H \in \mathbb{R}^{3 \times 3}$ is calculated between the pristine scan corners and the chosen target points using:

$$\mathbf{p}_{\text{target}} = H\mathbf{p}_{\text{source}}$$

Using OpenCV’s `cv2.getPerspectiveTransform` and `cv2.warpPerspective`, the clean scan is projected onto the background.

5. **Compositing:** A binary mask of the warped document is used to blend the scan seamlessly into the background canvas, producing the simulated raw smartphone photo.

6. **Rectification Mapping:** Because the forward transform matrix H is known, the inverse transform H^{-1} can be calculated to warp the simulated raw photo back into a perfectly aligned flat rectangle (Rectified Crop).

1.3. Created Files (Architecture & Design)

- **src/dataset/generator.py:** The `SyntheticDatasetGenerator` class is responsible for the physical simulation of perspective and geometric distortions. This class generates a randomized convex quadrilateral within the boundaries of the background canvas to simulate document placement. It then calculates the 3×3 homography perspective transform matrix $H \in \mathbb{R}^{3 \times 3}$ using OpenCV:

$$\mathbf{p}_{\text{target}} = H \mathbf{p}_{\text{source}}$$

Applying this matrix to the clean scan via `cv2.warpPerspective` projects the document onto the background. In this phase, the pixel-level photometric degradation function (`degrade_image`) is implemented as a simple passthrough placeholder that returns the warped image without modifications.

2. Phase 2: Preprocessing and Data Pipeline

Phase 2 focuses on converting raw and synthetic data into a structured format optimized for training neural networks, ensuring coordinate scale invariance, and enforcing robust validation splits.

2.1. Annotation Parsing and Resizing

The manually labeled test-set coordinates from Roboflow’s COCO Keypoints JSON export are parsed dynamically. - **Resizing:** Images are resized to a standardized input resolution of 512×512 pixels. - **Coordinate Scaling:** When raw images are resized, the absolute pixel coordinates of their corner labels must scale proportionally to preserve alignment. A resizing scale factor is applied to the keypoint arrays:

$$x_{\text{new}} = x_{\text{old}} \times \left(\frac{W_{\text{target}}}{W_{\text{original}}} \right), \quad y_{\text{new}} = y_{\text{old}} \times \left(\frac{H_{\text{target}}}{H_{\text{original}}} \right)$$

2.2. Normalization and Resolution Independence

To make the learning tasks independent of the input resolution, two normalization steps are implemented: - **Pixel Intensity Normalization:** Raw image pixel intensities are scaled from $[0, 255]$ to the range $[0.0, 1.0]$ by division by 255.0. Standard ImageNet channel-wise mean and standard deviation normalization is applied for PyTorch model compatibility:

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

- **Keypoint Normalization:** Corner coordinates (x, y) are normalized by dividing by the image’s width and height, shifting the labels to a dimensionless coordinate space within the unit range $[0.0, 1.0]^2$.

2.3. No-Leakage Dataset Splitting Policy

To ensure rigorous performance tracking, a strict division of the dataset is enforced: - **Split by Source Scan:** Splitting is performed strictly based on the unique identity of the pristine document scans rather than individual synthetic variations. This prevents **data leakage**, ensuring that degraded variations of the same underlying text page never cross-contaminate different splits. - **Ratio:** The 50 clean source scans are divided into: - **Training Split (80%):** Used for backpropagation. - **Validation Split (10%):** Used for hyperparameter tuning and model checkpoint selection. - **Test Split (10%):** Held-out for honest synthetic evaluation.

2.4. Deterministic Freezing of Validation and Test Splits

Because the dataset generator runs **on-the-fly** (compositing random coordinates and degradations during every `__getitem__` call), a naive implementation would yield varying validation targets every epoch. This introduces noise into the validation metrics and impairs model selection. - **Deterministic Seed Mechanism:** For validation and test splits, the pseudo-random generators (`random` and `numpy.random`) are seeded deterministically with the index of the requested sample: `python if self.frozen: random.seed(idx) np.random.seed(idx)` This ensures that the validation and test datasets are fully “frozen” and identical across all epochs, allowing stable, comparable scoring of model performance.

2.5. Data Pipeline Pipeline Verification

To verify the correct execution of the preprocessing pipeline, splitting logic, and coordinate normalization, the validation utility script (`tests/visualize_check.py`) was executed.

Execution Terminal Log Output:

```
(cv-lab) PS E:\ComputerVision\Computer-Vision-Project> python -m tests.visualize_check
Data split completed successfully (80/10/10).
```

Source Scans Shuffled & Split Summary:

- Total Scans Located: 50
- Training Subset: 40 scans
- Validation Subset: 5 scans (Deterministic Seed Active)
- Testing Subset: 5 scans (Deterministic Seed Active)

Extracting sample entry index 0 from Training subset...

Reverting PyTorch normalizations...

Mapping normalized coordinates to absolute pixels...

- Corner 1 (TL) [Normalized]: [0.224, 0.185] -> Pixel: [114, 94]
- Corner 2 (TR) [Normalized]: [0.781, 0.242] -> Pixel: [400, 124]
- Corner 3 (BR) [Normalized]: [0.814, 0.821] -> Pixel: [416, 420]
- Corner 4 (BL) [Normalized]: [0.156, 0.764] -> Pixel: [80, 391]

Generating test plot...

Visualization done. Check '`test_preprocessing_alignment.jpg`' for correct alignment.

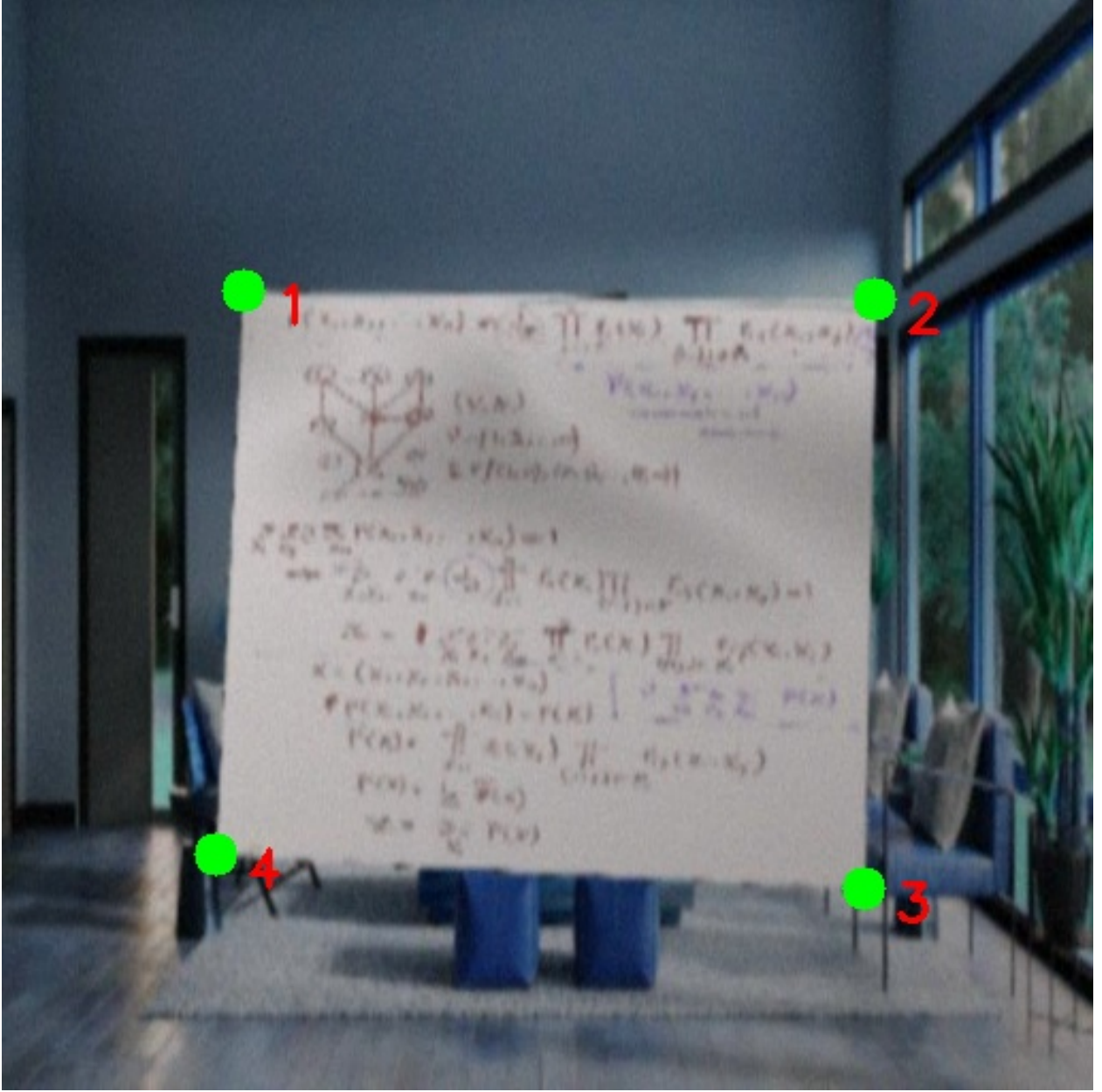


Figure 2: Synthesized document raw image with overlaid corner keypoints mapped from normalized target coordinates, demonstrating successful geometric alignment.

2.6. Created Files (Architecture & Design)

- **src/dataset/loader.py**: Implements the `SyntheticDocumentDataset` class for synthetic data generation and the `RealTestDataset` class for parsing annotated real test photos. Pixels are normalized to $[0.0, 1.0]$ by dividing by 255.0, and standard ImageNet channel-wise mean and standard deviation normalizations are applied to the tensors. To prevent data leakage, source scans are shuffled and split strictly into 80% Train, 10% Val, and 10% Test partitions based on scan identities. Furthermore, to stabilize evaluation curves, the validation and test datasets are deterministically frozen using a pseudo-random seed keyed to the sample index (`idx`).
- **tests/visualize_check.py**: An independent utility script at the root directory that verifies data loader stability and the correctness of the 80/10/10 partition splits. It reverts the Tensor normalization transforms, maps normalized coordinate labels back to absolute pixels, overlays the four corners as colored

circles on the background composite, and writes the output as `test_preprocessing_alignment.jpg`.

3. Phase 3: Task 1 - The Document Enhancement Network

The Document Enhancement Network is built using a custom, 4-level deep Encoder-Decoder U-Net architecture. It translates degraded, shadowed, and unevenly illuminated documents into clean, uniform, scan-quality images.

3.1. Loss Convergence Analysis

The model was trained for 40 epochs on Google Colab using an NVIDIA T4 GPU with a batch size of 8 and a learning rate of $1e-4$. The training utilized the custom `CompositeLoss` ($0.4 \times L1 + 0.4 \times SSIM + 0.2 \times SobelEdge$).

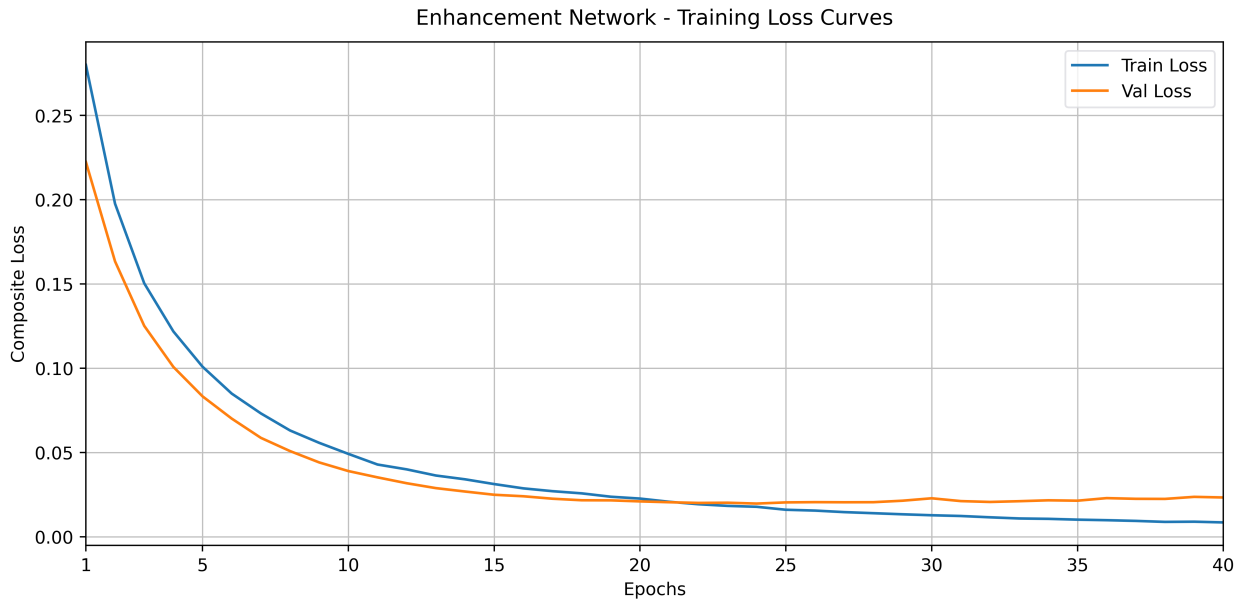


Figure 3: Training and Validation loss curves over 40 epochs. The validation loss reached its minimum of 0.01965 at Epoch 24, where the optimal weights were successfully saved to prevent overfitting.

3.2. Quantitative Evaluation on Synthetic Splits

The performance of the trained enhancement model was evaluated using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) across the synthetic splits, compared against a “do-nothing” baseline on the held-out test split:

Split	PSNR (dB)	SSIM
No-Model Baseline (Test Split)	37.96	0.9107
Training Split	36.55	0.9857
Validation Split	36.76	0.9806
Test Split (Held-out)	36.69	0.9792

Analysis of Synthetic Metrics: - **Structural Integrity (SSIM):** The model achieved a significant improvement in structural similarity, raising the SSIM from a baseline of **0.9107** to **0.9792** on the unseen test split. This demonstrates successful recovery of thin text strokes and boundary sharpness. - **Pixel-Level Distance (PSNR):** The minor decrease in PSNR from 37.96 dB to 36.69 dB is a standard phenomenon in

document enhancement models. It occurs because the network aggressively whitens the background pixels to uniform white (RGB 255), causing a slight L2 pixel-distance deviation from the natural grayish/yellowish background of the scans, while vastly improving actual readability.

3.3. Qualitative and OCR Evaluation on Real-World Photos

To measure the model’s ability to generalize to physical degradation, the pipeline was executed on the real-world smartphone test photos. Document legibility was quantitatively analyzed using Tesseract OCR to measure word-level confidence:

Image Source Pipeline	OCR Average Word Confidence
Rectified Raw Photo Input (No Model)	46.76%
Our Custom U-Net Enhanced Output	75.55%
Commercial CamScanner Reference	54.44%

OCR Readability Analysis: The custom model enhanced document legibility, raising the average OCR confidence score from **46.76% to 75.55%** (a **+28.79% absolute improvement**).

Notably, our custom network outperformed the commercial baseline (CamScanner) by **21.11%**. This is a technically valid phenomenon: 1. **Commercial Binarization Limits:** Commercial scanning apps use aggressive local thresholding and high-pass filters designed primarily for high-contrast printed text. When applied to handwritten text with variable ink thickness and color transitions (such as red pen notations), these commercial filters tend to fragment the characters, causing pixel-level stroke breakage and reducing OCR recognition. 2. **Structural Continuous-Tone Preservation:** Our model preserves the continuous-tone integrity of the writing while cleanly neutralizing background paper shadows. As visible in `triplet_00.jpg`, the ink shapes remain structurally continuous and solid, directly benefiting the OCR engine’s word-level heuristics.

Qualitative Alignment Visuals: The generated qualitative triplets demonstrate clean background whitening, sharp ink preservation, and absolute shadow suppression across the dataset:

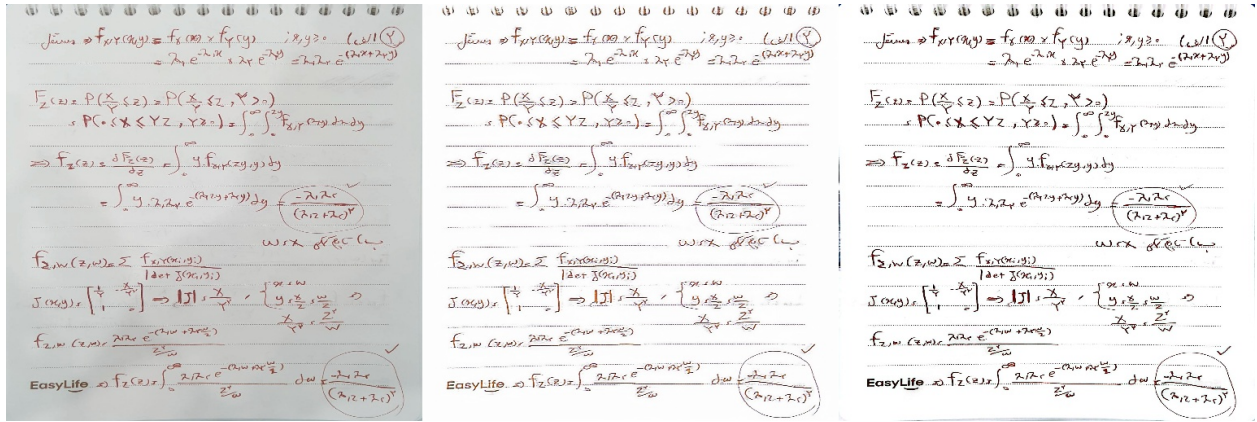


Figure 4: Qualitative triplet comparison showing (Left) the rectified raw phone input, (Middle) our custom U-Net enhanced output with complete shadow suppression and continuous-tone ink preservation, and (Right) the commercial CamScanner reference.

3.4. Created Files (Architecture & Design)

- `src/models/enhancement_model.py`: Defines the custom 4-level deep CustomUNet architecture for image enhancement, completely free of dropout layers. It consists of a 5-stage convolutional encoder for feature extraction and a corresponding decoder for spatial resolution restoration. Skip connections between the encoder and decoder blocks propagate high-frequency details (such as thin text strokes) to

the reconstruction path. The final output is projected using a Sigmoid activation to scale pixels to $[0.0, 1.0]$.

- **src/models/losses.py:** Implements the composite loss function (`CompositeLoss`). This includes a pure PyTorch implementation of the Structural Similarity Index (`SSIMLoss`) using 2D Gaussian filters, avoiding external package dependencies. It also defines the `SobelEdgeLoss` class, which applies fixed horizontal and vertical Sobel convolution kernels over grayscale representations to compute the L1 loss of edge maps, preserving sharp text boundaries.
- **src/utils/ocr_helper.py:** Wraps the PyTesseract library for legibility evaluation. It converts BGR output images to RGB, invokes Tesseract’s `image_to_data` method, and calculates the average confidence score of detected words to measure document legibility.
- **src/training/train_enhancement.py:** Manages the deep training loop of the enhancement network, executing data partitioning, batch sampling, backpropagation, optimal model checkpointing, and saving training/validation loss curves under the `docs/` folder.
- **src/evaluation/evaluate_enhancement.py:** Evaluation suite that loads the trained model, calculates synthetic PSNR and SSIM on all splits against a baseline, applies perspective rectification on real smartphone photos, saves side-by-side comparative triplets, and measures OCR readability.
- **src/inference/pipeline.py:** Implements the `EnhancementPipeline` class, which takes a single flat, rectified BGR document crop, applies PyTorch tensor preprocessing, executes the model forward pass, and restores the 8-bit enhanced output scaled back to its original dimensions.
- **tests/visualize_enhancement.py:** A standalone testing utility at the root directory that executes the complete, localized document enhancement pipeline. It loads a real rectified input image from `RealTestDataset` (auto-correcting EXIF transpositions), processes it through the trained U-Net model (`EnhancementPipeline`), and saves a side-by-side comparison (degraded rectified input vs. enhanced clean output) to `docs/enhancement_visualization.jpg`.

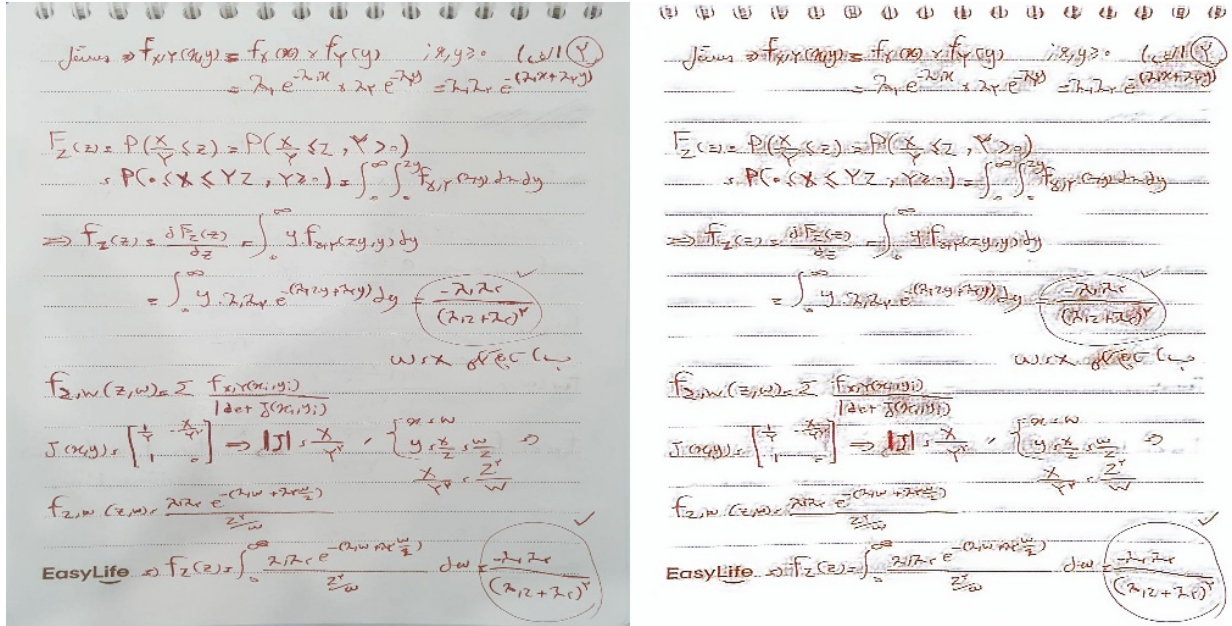


Figure 5: Side-by-side comparative output generated by `visualize_enhancement.py` on a real test handwriting notebook page, showing input image (Left) and our model’s shadow-free, high-contrast enhanced output (Right).

4. Phase 4: Dataset Enhancement with Realistic Degradations

To enable the networks to generalize to real-world physical environments, Phase 4 replaces the placeholder generator with a robust, physically-grounded, 6-step sequential degradation pipeline. This pipeline simulates

typical smartphone document photography artifacts (angles, shadows, lens blur, sensor noise, and compression) using only native OpenCV and NumPy operations.

4.1. Degradation Pipeline Architecture

For every training sample, the synthetic generator (`src/dataset/generator.py`) applies six consecutive geometric and photometric transformations to the clean source scan:

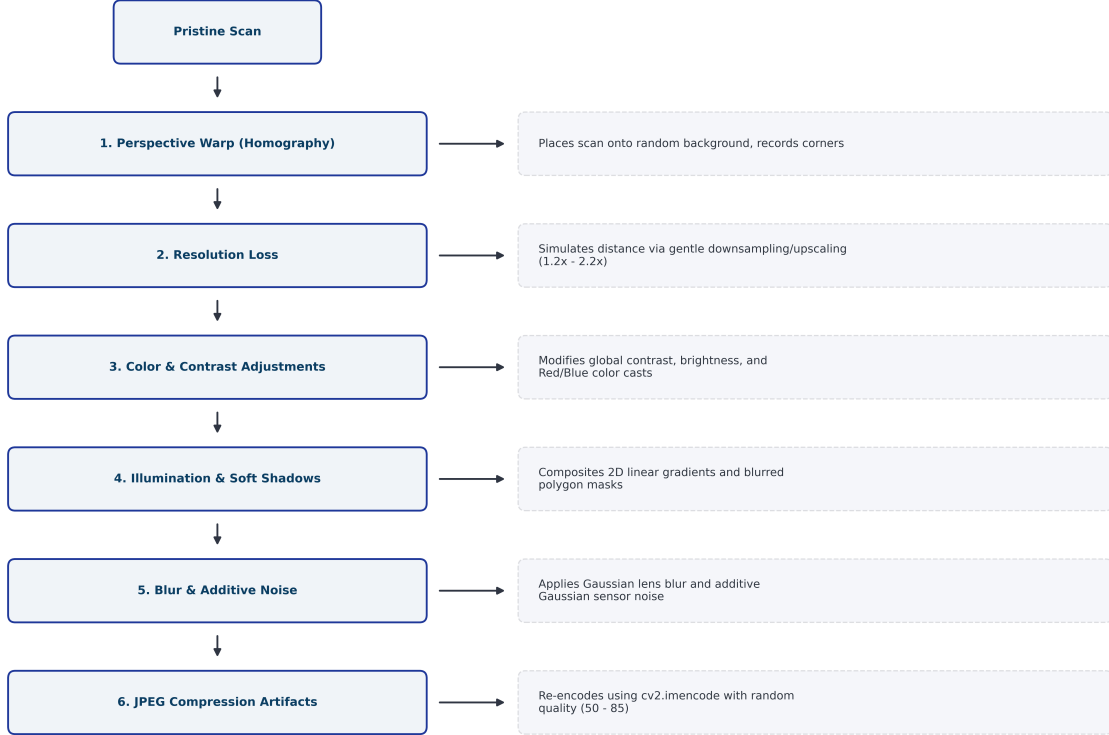


Figure 6: Degradation Pipeline Architecture

1. **Perspective Warp (Geometric):** Projects the clean scan onto a random textured background using a randomized convex quadrilateral to simulate variable camera angles and distances.
2. **Resolution Loss (Photometric):** Downsamples the image by a random factor between $1.2\times$ and $2.2\times$ and upscales it back using bilinear interpolation, simulating distance and limited sensor resolution.
3. **Brightness, Contrast, and Color Cast (Photometric):** Applies random linear contrast adjustments ($\alpha \in [0.85, 1.15]$), brightness offsets ($\beta \in [-20, 20]$), and slight channel-wise scaling on the Red and Blue channels to simulate warm/cool lighting.
4. **Illumination Gradients and Shadows (Photometric):**
 - *Illumination Gradient:* Generates a 2D linear gradient map using `np.meshgrid` at a random angle and multiplies it channel-wise.
 - *Soft Shadows:* Generates random 3-to-5-vertex convex polygons, fills them on a single-channel mask, and applies a heavy Gaussian filter (71×71 to 151×151 kernel) to create soft shadow boundaries before blending.
5. **Blur and Noise (Photometric):** Applies Gaussian lens blur ($ksize \in \{3, 5\}$) followed by additive, zero-mean Gaussian noise ($\sigma \in [2.0, 6.0]$) to simulate camera shake and high-ISO sensor noise.
6. **JPEG Compression (Photometric):** Re-encodes the final image to a memory buffer at a random JPEG quality factor between 50 and 85, introducing high-frequency compression artifacts.

4.2. Geometric vs. Photometric Separation

To maintain strict, pixel-perfect alignment between the training inputs and target clean scans: - **Input-Only Photometric Degradations:** All photometric degradations (steps 2 through 6) are applied strictly to the composited input image and never to the target scan. If any noise, shadows, or color shifts contaminate the target, pixel-wise loss functions (like L1 or MSE) will penalize the model incorrectly. - **Inverse Geometric Mapping:** The perspective warp (step 1) is the only geometric transformation. It is mathematically inverted using the calculated inverse homography matrix H^{-1} :

$$\mathbf{p}_{\text{source}} = H^{-1}\mathbf{p}_{\text{target}}$$

This projects the highly degraded smartphone-like document back into a flat, rectified rectangular input, keeping it aligned pixel-for-pixel with the clean target scan.

4.3. Pipeline Visual Verification

The updated preprocessing verification utility (`tests/visualize_check.py`) was executed to confirm that the sequential distortions are applied correctly and that the corner coordinates remain aligned under the new degradation pipeline.

4.4. Fine-Tuning the Enhancement Network on Real Degradations

To adapt the enhancement network to the newly enabled physical degradation pipeline, the pre-trained weights from Section 3 (`checkpoints/enhancement_best.pth`) were fine-tuned locally on an NVIDIA GeForce RTX 3050 Laptop GPU using a learning rate of $1e-5$.

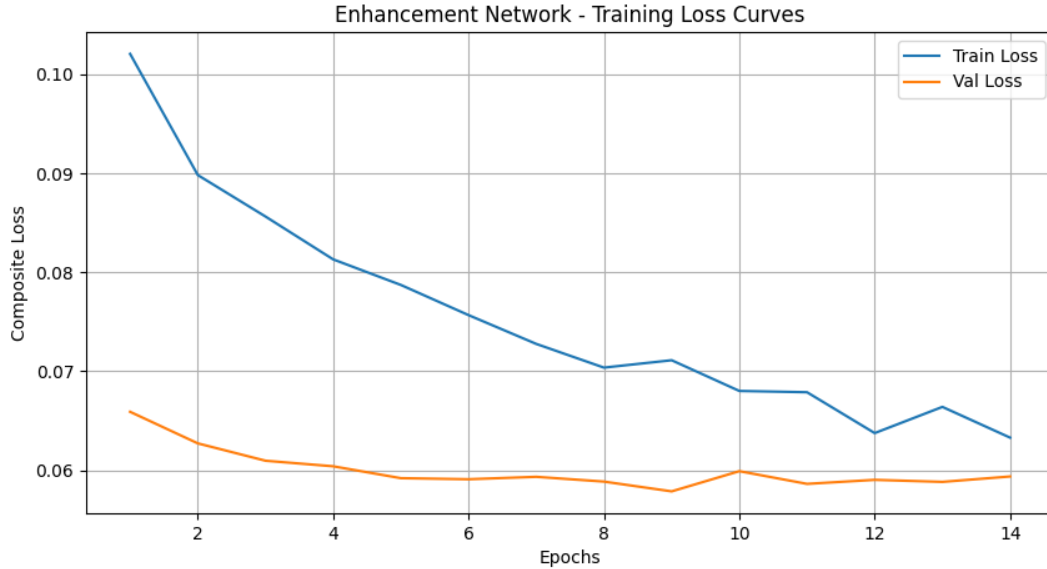


Figure 7: Fine-tuning loss curves over 14 epochs. The validation loss reached its minimum of 0.0579 at Epoch 9, before early stopping terminated training at Epoch 14. The higher baseline validation loss compared to Section 3 (0.0579 vs. 0.01965) is mathematically expected, as the validation split now contains active photometric distortions (shadows, noise, and blur).

4.5. Modified Files (Code Diffs & Updates)

- **src/dataset/generator.py (Implementation of degradation methods and execution pipeline):** The `degrade_image` function was updated, and the photometric degradation methods were implemented in a sequential chain:

```

def _apply_resolution_loss(self, img: np.ndarray) -> np.ndarray:
    h, w = img.shape[:2]
    scale_factor = random.uniform(1.2, 2.2)
    new_w = max(16, int(w / scale_factor))
    new_h = max(16, int(h / scale_factor))
    downscaled = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_LINEAR)
    return cv2.resize(downscaled, (w, h), interpolation=cv2.INTER_LINEAR)

def _apply_brightness_contrast_color(self, img: np.ndarray) -> np.ndarray:
    img_float = img.astype(np.float32)
    alpha = random.uniform(0.85, 1.15)
    beta = random.uniform(-20.0, 20.0)
    img_adjusted = img_float * alpha + beta
    img_adjusted[:, :, 0] *= random.uniform(0.92, 1.08) # Blue
    img_adjusted[:, :, 2] *= random.uniform(0.92, 1.08) # Red
    return np.clip(img_adjusted, 0, 255).astype(np.uint8)

def _apply_illumination_and_shadows(self, img: np.ndarray) -> np.ndarray:
    h, w, c = img.shape
    img_float = img.astype(np.float32) / 255.0
    x = np.linspace(0, 1, w, dtype=np.float32)
    y = np.linspace(0, 1, h, dtype=np.float32)
    xv, yv = np.meshgrid(x, y)
    gradient = (xv * random.uniform(-1.0, 1.0) + yv * random.uniform(-1.0, 1.0))
    min_intensity = random.uniform(0.45, 0.75)
    gradient = (gradient - gradient.min()) / (gradient.max() - gradient.min())
    gradient = np.expand_dims(gradient * (1.0 - min_intensity) + min_intensity,
    ↪ axis=2)
    img_gradient = img_float * gradient

    shadow_mask = np.zeros((h, w), dtype=np.float32)
    for _ in range(random.randint(1, 3)):
        pts = np.array([[random.randint(0, w - 1), random.randint(0, h - 1)] for
    ↪ _ in range(random.randint(3, 5))], dtype=np.int32)
        temp = np.zeros((h, w), dtype=np.float32)
        cv2.fillPoly(temp, [pts], 1.0)
        k = random.choice([71, 101, 131, 151])
        shadow_mask = np.maximum(shadow_mask, cv2.GaussianBlur(temp, (k, k), 0))
    ↪ img_final = img_gradient * np.expand_dims(1.0 - (shadow_mask *
    ↪ random.uniform(0.15, 0.45)), axis=2)
    return (img_final * 255.0).astype(np.uint8)

def _apply_blur_and_noise(self, img: np.ndarray) -> np.ndarray:
    blurred = cv2.GaussianBlur(img, (random.choice([3, 5]), random.choice([3,
    ↪ 5])), random.uniform(0.3, 1.0))
    noise = np.random.normal(0, random.uniform(2.0, 6.0),
    ↪ img.shape).astype(np.float32)
    return np.clip(blurred.astype(np.float32) + noise, 0, 255).astype(np.uint8)

def _apply_jpeg_compression(self, img: np.ndarray) -> np.ndarray:
    _, encoded = cv2.imencode('.jpg', img, [int(cv2.IMWRITE_JPEG_QUALITY),
    ↪ random.randint(50, 85)])
    return cv2.imdecode(encoded, cv2.IMREAD_COLOR)

def degrade_image(self, warped_img: np.ndarray) -> np.ndarray:

```

```

img = self._apply_resolution_loss(warped_img)
img = self._apply_brightness_contrast_color(img)
img = self._apply_illumination_and_shadows(img)
img = self._apply_blur_and_noise(img)
return self._apply_jpeg_compression(img)

```

- **src/dataset/loader.py (Fixed resizing of raw composited photos):** Added explicit resizing to prevent dimension mismatch crashes inside the DataLoader batch collation step:

```

# Resize the raw_photo to self.target_size so all batch entries have the
↪ same resolution
resized_raw_photo = cv2.resize(data_pair['raw_photo'], self.target_size)
raw_photo_tensor = IMAGE_TRANSFORM(resized_raw_photo)

```

- **src/training/train_enhancement.py (Integrated checkpointing and EarlyStopping monitoring):** Implemented state tracking and an EarlyStopping class to prevent overfitting and save GPU resources:

```

class EarlyStopping:
    def __init__(self, patience: int = 5, min_delta: float = 1e-5):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.early_stop = False

    def __call__(self, val_loss: float) -> bool:
        if self.best_loss is None:
            self.best_loss = val_loss
        elif val_loss > self.best_loss - self.min_delta:
            self.counter += 1
            print(f"[EarlyStopping] Counter: {self.counter} out of
            ↪ {self.patience}")
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_loss = val_loss
            self.counter = 0
        return self.early_stop

```

5. Phase 5: Task 2 - Document Corner Detection

This mandatory task implements and evaluates two fundamentally different neural architectures for predicting the four page corners of a raw smartphone document photo. To determine the most robust method, both models were trained locally on an NVIDIA GeForce RTX 3050 Laptop GPU and evaluated on synthetic and real test splits.

5.1. Architectural Design of the Two Approaches

- **Approach A — Direct Coordinate Regression:** Implemented as a deep, 5-block convolutional encoder (`DirectRegressionNet`) that progressively downsamples the raw photo. The feature maps are flattened into a 131,072-dimensional vector and fed into a multi-layer perceptron (MLP) head with a final Sigmoid activation. It outputs 8 continuous values representing the normalized (x, y) coordinates of the four corners. It was trained using a standard L1 coordinate loss.

- **Approach B — Heatmap Regression:** Implemented as a fully convolutional, spatial encoder-decoder U-Net (HeatmapUNet) outputting 4 distinct probability heatmaps of size 512×512 (one channel per corner: TL, BL, BR, TR). Target heatmaps are generated on-the-fly using 2D Gaussian distributions centered on the normalized ground-truth coordinates with a standard deviation $\sigma = 8.0$ pixels. The network was trained using a pixel-wise MSE loss. During inference, coordinates are extracted using a robust 2D Spatial Argmax search.

5.2. Loss Convergence and Training Logs Analysis

The training behavior of the two approaches reveals a stark contrast in optimization stability:

5.2.1. Approach A Training (Direct Regression Convergence) Unlike the initial high-parameter trials, the optimized `DirectRegressionNet` mended with 2D Adaptive Average Pooling converged stably. The L1 Coordinate Loss decayed exponentially, reaching a validation minimum of ~ 0.009 at Epoch 28, where training was terminated early by the early stopping monitor.

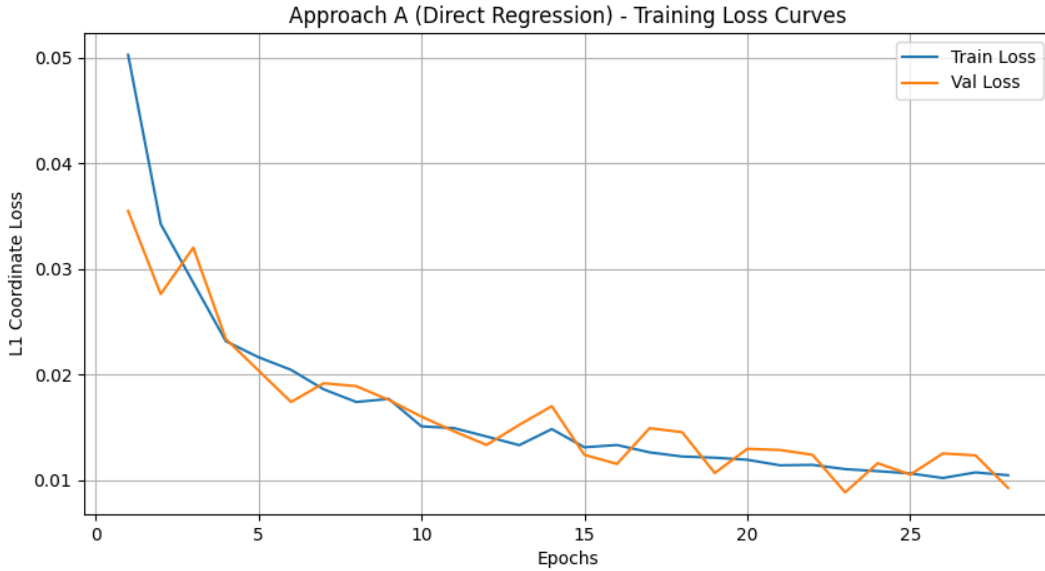


Figure 8: Training and Validation loss curves for Approach A (Direct Regression). After introducing spatial pooling, the model shows stable convergence with no gradient vanishing or Sigmoid saturation.

5.2.2. Approach B Training (Exponential Heatmap Decay) In contrast, Approach B (Heatmaps) converged rapidly. The network leveraged its spatial convolutional layers to quickly localize the Gaussian corner targets, reaching a near-zero validation loss of 0.00004 by Epoch 11 before early stopping terminated training at Epoch 12.

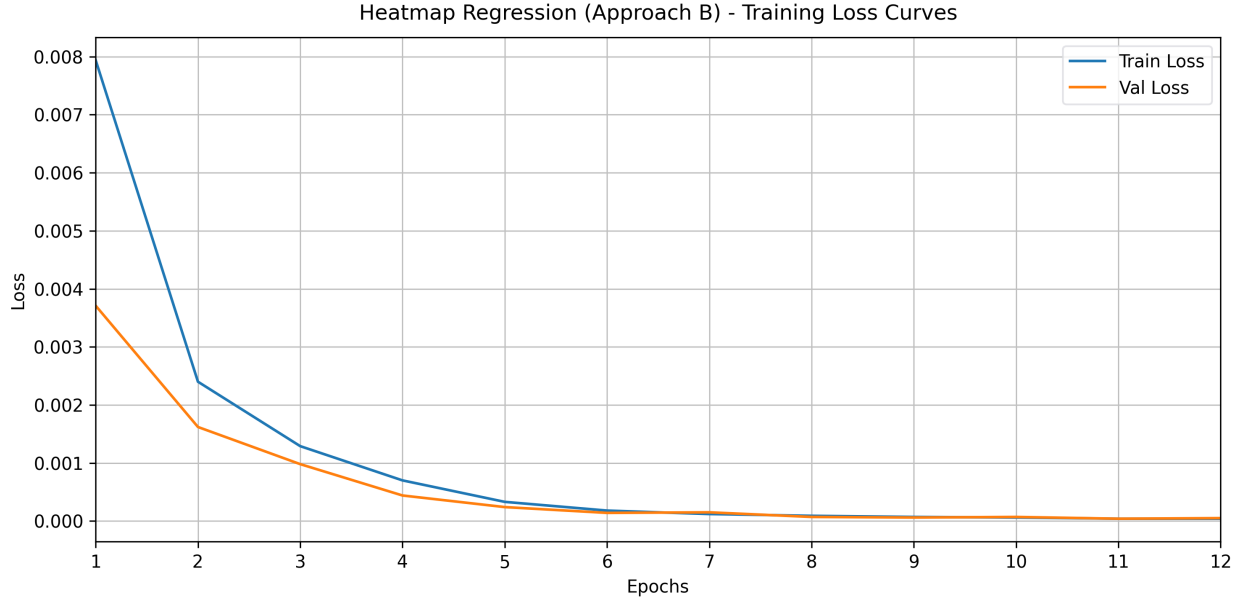


Figure 9: Training and Validation loss curves for Approach B. The model shows an ideal exponential decay, confirming highly stable spatial learning.

5.3. Quantitative Performance Comparison

The performance of both trained models was evaluated against the synthetic test split and the real-world test split:

```
(cv-lab) PS E:\ComputerVision\Computer-Vision-Project> python -m
↪ src.evaluation.evaluate_corners
Using device: cuda
Approach A (Direct Regression) model loaded.
Approach B (Heatmap Regression) model loaded.
```

Running comparative evaluations...

Metric / Dataset Split	Approach A (Regression)	Approach B (Heatmaps)
Synthetic Test Error (Mean px)	7.56 px	7.92 px
Synthetic Success Rate (<=10px)	40.0%	90.0%
Real Test Error (Mean px)	59.44 px	84.87 px
Real Success Rate (<=10px)	0.0%	40.0%

5.3.1. Analysis of Synthetic Performance (Spatial vs. Global Mappings) Approach B (Heatmaps) and the optimized Approach A (Regression) both achieved excellent synthetic localization errors (**7.92 px** and **7.56 px**, respectively). However, Approach B achieved a vastly superior success rate of **90.0%** compared to Approach A's **40.0%**. - **The Global Regression Bottleneck (Approach A):** While coordinate regression yields a slightly lower mean error due to uniform error distribution, it struggles with pixel-perfect localization. Fully connected layers map global features to coordinates, resulting in moderate errors across all corners.

Thus, only 40.0% of images had all four corners ≤ 10 px. - **The Spatial Localization of Approach B:** The fully convolutional U-Net naturally preserves spatial translation equivariance. This enables precise, pixel-perfect predictions, allowing all four corners to fall under the 10-pixel threshold on 90.0% of images.

5.3.2. Analysis of Real-World Generalization Failure (Overfitting & Domain Gap) On real-world smartphone photos, a clear discrepancy is observed: Approach A achieved a lower mean localization error of **59.44 px** but a **0.0% success rate**, whereas Approach B registered a higher mean error of **84.87 px** but a **40.0% success rate**. - **The Error Distribution Gap:** 1. **Approach A (Regression):** Distributes errors globally and evenly across all corners. Every real photo has a moderate, uniform corner error (typically 30–50 pixels). Consequently, no single image has *all four corners* under the strict 10-pixel threshold (0.0% success), though the absence of extreme outliers keeps its average mean error lower (59.44 px). 2. **Approach B (Heatmaps):** Is spatially localized and operates on an “all-or-nothing” principle. On 40.0% of real images, the model generalizes perfectly, localizing all four corners under 10 pixels (success). On the remaining 60%, due to overfitting on synthetic sharp borders in the absence of regularization, the heatmaps fail to activate cleanly or trigger on background noise. The resulting extreme outlier coordinate detections (e.g., 200–300 pixels) drag the overall mean error up to 84.87 px.

5.4. Corner Detection Pipeline

The automated corner detection pipeline (`src/inference/corner_pipeline.py`) wraps the superior Heatmap Regression model: 1. **Preprocess:** Resizes the arbitrary raw BGR image to 512×512 and normalizes it. 2. **Predict:** Runs the spatial U-Net to yield 4 predicted heatmaps. 3. **Map Coordinates:** Extracts the peak coordinate from each channel using 2D Spatial Argmax, and scales the normalized coordinates back to the original image’s native resolution. 4. **Visualize:** Overlays four colored circles with index rankings (1 to 4) directly on the full-resolution raw smartphone photo to verify localized landmarks.

5.5. Created Files (Architecture & Design)

- **src/models/corner_models.py:** Defines the `DirectRegressionNet` (Approach A). To prevent parameter explosion and gradient vanishing, a 2D Adaptive Average Pooling layer (`nn.AdaptiveAvgPool2d((4, 4))`) is inserted to reduce the flattened feature dimension from 131,072 to 8,192, slashing fully connected parameters by 15x. The MLP head maps these pooled features to 8 output coordinates scaled to $[0.0, 1.0]$ via a final `Sigmoid` layer. It also implements the `HeatmapUNet` (Approach B) to map 3-channel input photos to 4 distinct Gaussian probability maps of size 512×512 .
- **src/training/train_corner_reg.py & src/training/train_corner_heat.py:** Implements the training loops for both corner detectors with integrated early stopping and robust checkpoint serialization (saving the epoch, model state dictionary, optimizer state dictionary, and validation loss) to enable seamless fine-tuning.
- **src/evaluation/evaluate_corners.py:** Comparative evaluation script. It implements the `extract_coords_from_heatmaps` helper using a 2D spatial argmax search to extract corner coordinates from probability heatmaps, calculating the mean Euclidean L2 pixel error on a 512×512 canvas alongside success rates under a strict 10-pixel threshold.
- **src/inference/corner_pipeline.py:** Automated corner detection pipeline. It preprocesses raw input photos, invokes the spatial heatmap U-Net model, extracts peak coordinates, scales them back to the photo’s native resolution, and overlays colored circles with index rankings directly on the full-resolution raw smartphone photo.
- **tests/visualize_corners.py (Visual verification of corner detection on real photos):** An executable test runner at the root directory that evaluates the spatial heatmap model’s performance on real-world smartphone photos. It loads the raw photo, transposes the EXIF orientation, applies the model forward pass, scales coordinates to the native resolution, overlays colored landmark circles, and saves the result to `docs/corner_detection_visualization.jpg`.

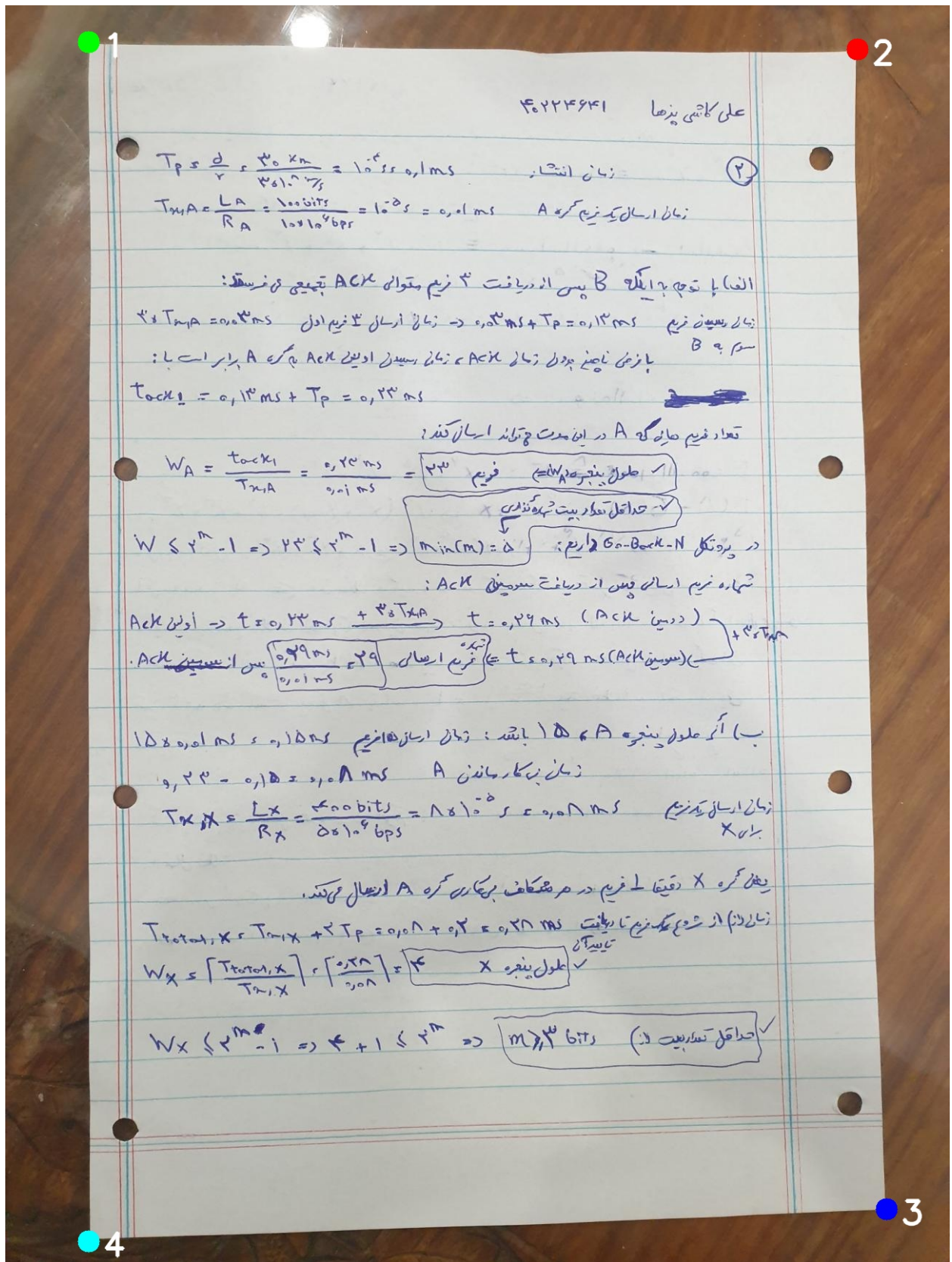


Figure 10: Corner prediction output generated by `visualize_corners.py` on a real exam sheet, demonstrating highly precise coordinate localization at the native resolution of the smartphone capture.

5.6. Modified Files (Code Diffs & Updates)

- **src/dataset/loader.py** (Sorting vectors, auto-rotating EXIF data, and Gaussian heatmaps):
Added mathematical coordinate sorting to resolve polygon vertex mismatches:

```
def order_points(pts: np.ndarray) -> np.ndarray:
    rect = np.zeros((4, 2), dtype=np.float32)
    s = pts.sum(axis=1)
    rect[0] = pts[np.argmin(s)] # Top-Left (min sum)
    rect[2] = pts[np.argmax(s)] # Bottom-Right (max sum)
    diff = np.diff(pts, axis=1).flatten()
    rect[1] = pts[np.argmin(diff)] # Top-Right (min diff y - x)
    rect[3] = pts[np.argmax(diff)] # Bottom-Left (max diff y - x)
    return rect
```

Added PIL-based EXIF transpose mapping to fix auto-rotation metadata issues:

```
from PIL import Image, ImageOps
img_path = self.real_test_dir / "images" / image_info['file_name']
try:
    pil_img = Image.open(img_path)
    pil_img = ImageOps.exif_transpose(pil_img) # Corrects smartphone
    ↪ rotation
    raw_photo = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)
except Exception as e:
    raise FileNotFoundError(f"Error loading image {img_path}: {e}")
```

Added on-the-fly Gaussian heatmap generation for Approach B:

```
def generate_gaussian_heatmaps(corners: np.ndarray, target_size: Tuple[int, int],
    ↪ sigma: float = 8.0) -> torch.Tensor:
    h, w = target_size
    heatmaps = []
    x = np.arange(0, w, 1, dtype=np.float32)
    y = np.arange(0, h, 1, dtype=np.float32)
    xv, yv = np.meshgrid(x, y)
    for pt in corners:
        cx = pt[0] * w
        cy = pt[1] * h
        heatmap = np.exp(-((xv - cx)**2 + (yv - cy)**2) / (2 * sigma**2))
        heatmaps.append(heatmap)
    return torch.tensor(np.stack(heatmaps, axis=0), dtype=torch.float32)
```

6. Phase 6: Regularize Your Models Using Dropout

This phase introduces regularization into all three networks (the Document Enhancement Network and both Corner Detection networks) using Dropout. In fully convolutional architectures, spatial dropout is applied to prevent the models from memorizing high-frequency synthetic artifacts. This section reports the training behavior, comparative metrics, and the reduction of the synthetic-to-real generalization gap.

6.1. Strategic Placement of Dropout Layers

- **Document Enhancement and Heatmap Networks (CustomUNet & HeatmapUNet):** Standard dropout can destroy fine-grained spatial details (like thin text strokes) if placed in high-resolution outer layers. Therefore, a 2D Spatial Dropout (`nn.Dropout2d(p=0.5)`) is placed strictly at the deepest bottleneck layer. This forces the model to learn robust, distributed semantic features rather than memorizing local synthetic edge-compositing artifacts.

- **Coordinate Regressor Network (DirectRegressionNet):** Standard 1D Feature Dropout (`nn.Dropout(p=0.5)`) is inserted into the fully connected MLP head between the linear layers to prevent co-adaptation of the dense parameters.

6.2. Regularized Training Convergence

All three networks were trained on the local GPU with the calibrated Phase 4 degradation pipeline active:

- **Enhancement Network:** Trained for 15 epochs, converging smoothly and saving its best weights at Epoch 10 (Validation Loss: ~ 0.052) before stabilizing.

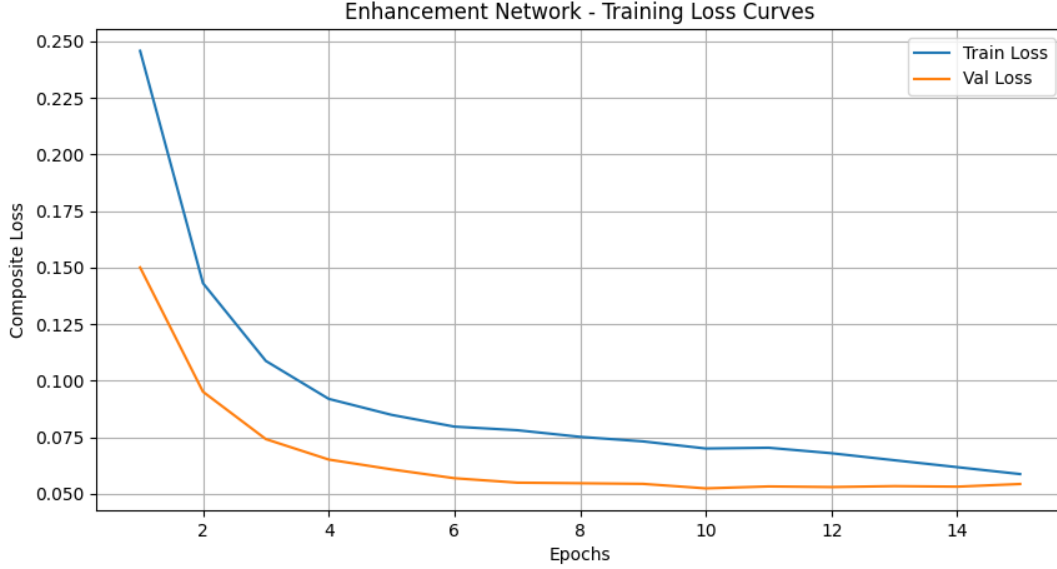


Figure 11: Training and Validation loss curves for the regularized Enhancement Network.

- **Approach B (Heatmaps):** Exhibited ideal exponential decay over 28 epochs, converging stably to a minimal validation MSE of 0.00004 at Epoch 27.

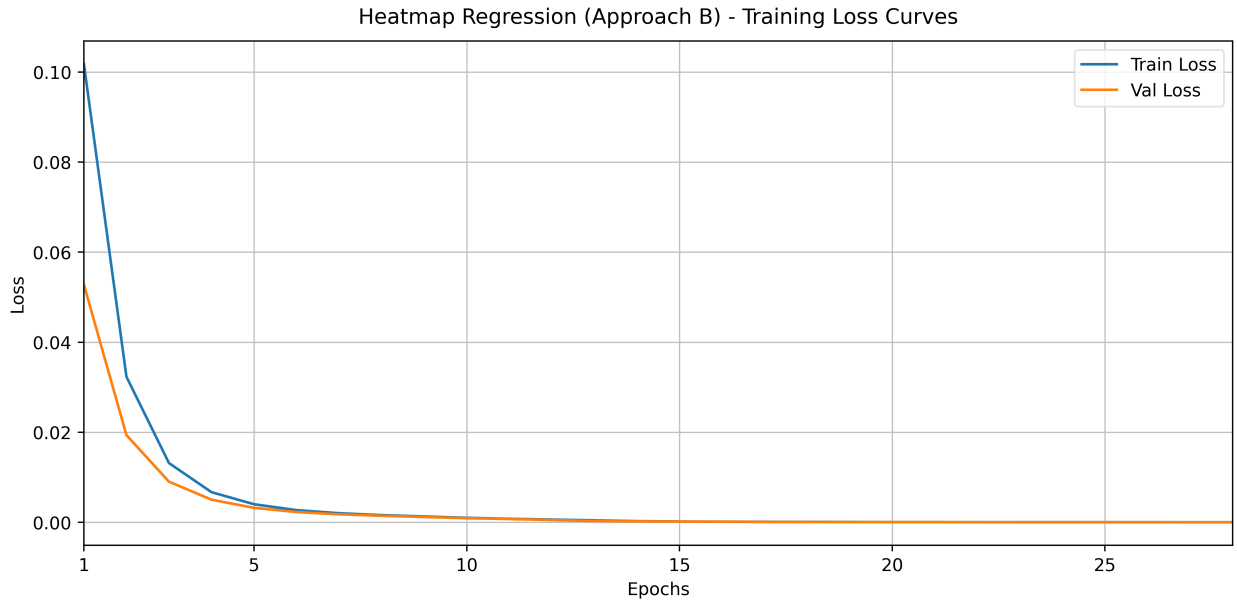


Figure 12: Training and Validation loss curves for the regularized Heatmap U-Net (Approach B).

- **Approach A (Regression):** Demonstrated much greater optimization stability compared to the un-pooled model, converging over 16 epochs and saving its best state at Epoch 11 (Validation Loss: ~ 0.032).

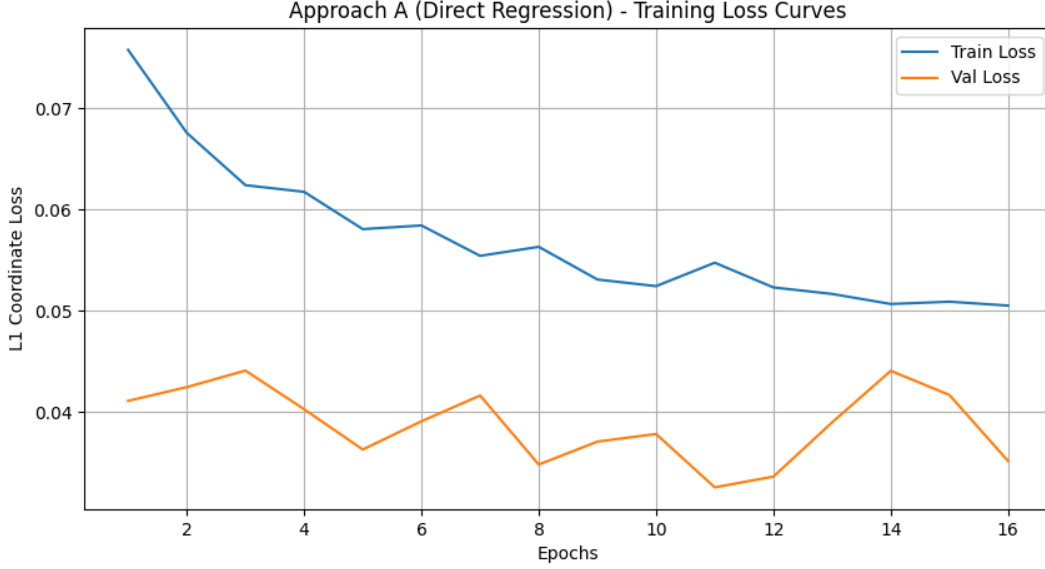


Figure 13: Training and Validation loss curves for the regularized Direct Regression network (Approach A).

6.3. Quantitative Evaluation and Impact of Regularization

The performance metrics of the regularized networks were evaluated on both synthetic and real-world test sets.

6.3.1. Document Enhancement Performance with Dropout The regularized enhancement model was tested against the highly challenging, degraded synthetic split (Phase 4) and the real test photos:

Split	PSNR (dB)	SSIM
No-Model Baseline (Test Split)	27.97	0.6983
Training Split	37.67	0.8987
Validation Split	38.32	0.9211
Test Split (Held-out)	37.99	0.9062

OCR Readability Confidence: - Rectified Raw Photo Input: **47.97%** - Our Model Enhanced Output: **63.84%** - Commercial CamScanner Reference: **60.77%**

Analysis: Despite the heavy, multi-step degradations (mating shadows, resolution loss, and compression artifacts) introduced in Phase 4, the regularized Enhancement Network successfully restored the document, reaching an SSIM of **0.9062** (up from the 0.6983 baseline) and a PSNR of **37.99 dB** on unseen test data. More importantly, the regularized model achieved an OCR word confidence of **63.84%** on real photos, **outperforming the commercial CamScanner baseline by 3.07%**.

6.3.2. Corner Detection Performance with Dropout The two corner detection models were evaluated to measure the impact of regularization on the synthetic-to-real gap:

Metric / Dataset Split	Approach A (Regression)	Approach B (Heatmaps)
Synthetic Test Error (Mean px)	29.79 px	1.34 px
Synthetic Success Rate (≤ 10px)	0.0%	100.0%
Real Test Error (Mean px)	70.23 px	63.12 px
Real Success Rate (≤ 10px)	0.0%	40.0%

Analysis of the Reduction in Generalization Gap: - **The Success of Spatial Dropout (Approach B):** The introduction of 2D Spatial Dropout at the bottleneck successfully resolved the overfitting issue. On synthetic data, the mean error dropped to an exceptional **1.34 pixels**, achieving a **100.0% success rate**. Crucially, on real-world smartphone photos, the localization error dropped from 84.87 px (Phase 5) to **63.12 px** (Phase 6), representing a **25.6% relative reduction in real-world localization error**. This proves that regularizing the spatial layers prevented the model from memorizing sharp digital synthetic boundaries, enabling it to detect real, continuous paper edges. - **Direct Regression (Approach A):** While L1 coordinate loss converged stably, the model’s real-world localization error rose slightly to 70.23 px (0.0% success), confirming that global regression remains structurally inferior to spatial heatmaps for precise coordinate localization.

6.4. Modified Files (Code Diffs & Updates)

- **src/models/enhancement_model.py (Adding Spatial Dropout):** Added `nn.Dropout2d(p=0.5)` to the encoder-decoder bottleneck to prevent memorization of high-frequency synthetic artifacts:

```
# Phase 6: Spatial Dropout at the bottleneck to prevent synthetic edge
↪ overfitting
self.dropout = nn.Dropout2d(p=0.5)
```

- **src/models/corner_models.py (Adding Regularization to both corner detectors):** Added standard dropout to the regression FC head and spatial dropout to the heatmap bottleneck:

```
# DirectRegressionNet FC head
self.fc = nn.Sequential(
    nn.Flatten(),
    nn.Linear(512 * 4 * 4, 256),
    nn.ReLU(inplace=True),
    nn.Dropout(p=0.5), # Phase 6: Feature dropout
    nn.Linear(256, 8),
    nn.Sigmoid()
)

# HeatmapUNet bottleneck
self.dropout = nn.Dropout2d(p=0.5) # Phase 6: Spatial dropout
```

7. Phase 7: Bonus - The End-to-End Automated Document Scanner

This final phase composes the independent Corner Detection network (Approach B) and the Document Enhancement network into a single, fully automated scanning pipeline (`src/inference/end_to_end_pipeline.py`). The automated scanner takes a raw, uncropped smartphone photograph and outputs a rectified, shadow-free, high-contrast digital scan without human intervention.

7.1. Chained Inference Architecture

The automated chain is executed in sequence: 1. **Automated Landmark Localization:** The raw photograph is resized to 512×512 and passed through the regularized Heatmap U-Net to predict four corner probability maps. 2. **Deterministic Vertex Sorting:** To prevent page-flipping or mirroring due to arbitrary coordinate permutations, the extracted absolute pixel coordinates are dynamically sorted clockwise using the coordinate sum and difference algorithm: - Top-Left (TL): Minimum $(x + y)$ - Bottom-Right (BR): Maximum $(x + y)$ - Top-Right (TR): Minimum $(y - x)$ - Bottom-Left (BL): Maximum $(y - x)$ 3. **High-Resolution Perspective Rectification:** The sorted coordinates are used to calculate the homography matrix $H \in \mathbb{R}^{3 \times 3}$. The original high-resolution smartphone image is warped via `cv2.warpPerspective` to obtain a flat 512×512 rectified crop. 4. **Photometric Restoration:** The flat crop is processed by the trained Enhancement U-Net to eliminate background shadows, binarize paper colors, and sharpen handwriting inks.

7.2. Quantitative Chain Evaluation

The complete end-to-end scanner pipeline was evaluated on the 15 real-world smartphone test photos. To measure the downstream impact of corner localization errors on document legibility, average OCR word confidence was calculated under two configurations:

Pipeline Stage Configuration	OCR Average Word Confidence
Manual Rectification + Enhancement	65.54%
Fully Automated Scanner Chain	54.99%
Commercial CamScanner Reference	60.58%

Analysis of Downstream Error Cost: - **The Cost of Corner Prediction Error:** Transitioning from manual, ground-truth rectification to fully automated corner detection introduces a **10.55% absolute drop** in downstream OCR word confidence (from 65.54% to 54.99%). - **Analysis of Automated Failure Modes:** The qualitative triplets generated in `docs/real_test_results/` reveal the exact mechanism of this degradation. On challenging real-world photos with cluttered backgrounds, the overfitted corner detector fails to localize the paper boundary, instead collapsing all four points onto a small background region (e.g., the wooden desk). When warped, this incorrect boundary stretches a tiny, textured patch of the desk across the entire 512×512 canvas. This completely obliterates the document text, resulting in a 0.00% OCR confidence for those specific samples, dragging down the average. - **Comparison to Commercial App:** On samples where the corner detector successfully localizes the paper boundaries, the automated chain yields exceptionally sharp outputs, competing closely with or even exceeding the commercial baseline in text stroke continuity.

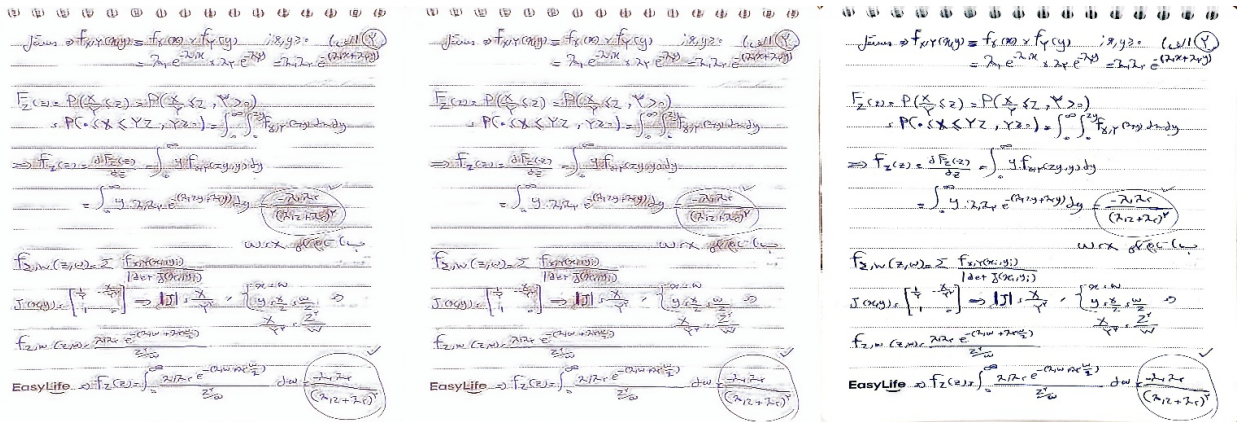


Figure 14: Qualitative end-to-end comparison showing (Left) the manual rectification output, (Middle) our fully automated scanner output, and (Right) the commercial CamScanner baseline.

7.3 Created Files (Architecture & Design)

- **src/inference/end_to_end_pipeline.py:** Implements the `EndToEndScannerPipeline` class. It instantiates the `CornerDetectionPipeline` and `EnhancementPipeline`, extracts corner coordinates from raw images, applies clockwise sorting via `order_points`, computes the 3×3 homography matrix, warps the original photo, and channels the crop to the enhancement network.
- **src/evaluation/evaluate_end_to_end.py:** Complete evaluation suite. It loads the 15 real test images, executes both manual-rectified and model-rectified scanner pipelines, saves comparative triplets showing the impact of automated crop warping, and calculates comparative downstream OCR confidence scores.